A measured refinement of a concise iPRA checker

A 513-byte implementation, its proof, and the unachieved complexity target

Research prepared for Vladimir Reshetnikov

12 November 2026

Abstract

The requested result is a presentation bi-interpretable with full first-order intuitionistic primitive recursive arithmetic whose complete checker improves the current repository frontier. This investigation does *not* establish such an improvement in all complexity coordinates and does not establish a new mathematical axiomatization. It establishes a smaller byte implementation of the existing guarded scalar presentation: 513 normalized UTF-8 bytes in place of 516, with 268 significant tokens and 306 AST nodes. The token and AST records, 256 and 278 respectively, are not improved. The transformation has a written successful-trace inclusion proof and preserves completeness for the canonical certificate compiler. A separately measured 1183-byte program validates the integer/list alphabet and guarded public goal. The inherited axioms, logical encodings, and whole-domain bi-interpretation are stated explicitly. All computational checks and comparison sources are supplied. The distinction between a byte improvement, an empirical coordinate envelope, and an actual mathematical simplification is maintained throughout.

Result status

Question	Established here?
Smaller matched compact byte implementation	Yes: 516 → 513 bytes.
Fewer than 256 significant tokens	No.
Fewer than 278 AST nodes	No.
One program below all three incumbent coordinates	No.
New arithmetic or logical axiom deletion	No.
Preservation of the existing intended theorem set	Yes, by the refinement theorem, relative to the pinned calculus.
New proof-assistant verification	No.

Thus the central requested research target remains open in this deliverable. The positive result is an implementation refinement with explicit limits.

Contents

1 The comparison is with a fixed current revision **3**
1.1 Reproduced measurements . . . 3
1.2 What the small size includes . . . 3

2 The actual refinement **4**
2.1 Root shape and false results . . . 4
2.2 A successful trace is not enlarged . . . 4
2.3 Canonical completeness survives the extra evaluation . . . 5

3 The inherited mathematical presentation, explicitly **6**
3.1 Terms, formulas, and semantics . . . 6
3.2 The complete rule inventory . . . 6
3.3 Equality, cases, and equation induction . . . 7

4 Recovering all intuitionistic logical operations **7**

5 The inherited whole-domain bi-interpretation **8**
5.1 Ordinary realization and the falsity bridge . . . 8
5.2 The arithmetic dependency order . . . 9
5.3 Both composites are proved internally . . . 10

6 The compact codec is a separate layer **10**
6.1 Complete compact program . . . 11
6.2 Why context and phase changes need proofs . . . 11

7 A barrier to removing every supplied term **12**

8 Executed evidence and its limits **12**

9 Further research questions **13**

A Using the archive **14**

B The validated standalone program **15**

1 The comparison is with a fixed current revision

The repository was inspected at commit `62396b06020f794fa6dc01b4025a6a5db9d4d28a`. Its current overview lists ninety research packages and a Lean development. The latest intake already includes introduction at the newest numerical parameter, the removal of a naming-pattern exception, conditional copy, and presentations with one numerical constructor. These results are existing work, not contributions of this report [1, 9, 10].

The stated metric counts the complete Python module after normalization by `ast.unparse`, followed by a newline, under CPython 3.12.14. It records significant tokens, AST nodes, and normalized UTF-8 bytes. Comments and formatting disappear under this normalization; an interpreter hidden in a string or an omitted validator would still be a substantive change of what is counted. The supplied measurement script uses the named constants from `tokenize`, because token-number assignments depend on Python version.

1.1 Reproduced measurements

Module	Tokens	AST nodes	Bytes
Pinned token record	**256**	283	594
Pinned AST record	260	**278**	602
Pinned byte record	265	306	516
New compact refinement	268	306	**513**
Matched strict control	459	524	1186
New strict refinement	462	524	1183

The first three rows are three different programs, supplied unchanged in `reference/checkers/`. The compact refinement is about 0.58% shorter in normalized bytes than the byte control, uses three more tokens, and has the same AST-node count. Relative to the coordinate envelope it is twelve tokens and twenty-eight nodes above the two other records.

Remark 1.1 (An empirical envelope is not an impossibility theorem)**.** The achieved vector $(256, 278, 516)$ is a coordinatewise envelope of existing implementations. It supplies upper bounds on the unknown optima of the three minimization problems. It is not a proved lower bound on every possible presentation of iPRA, and it is not the size vector of one existing program. No claim of an absolute simplest axiomatization follows from a finite collection of checkers.

1.2 What the small size includes

The 513-byte and 516-byte modules use the same multiline JSON input and `ACCEPT\string \n` output convention. They include the opener, rule checking, context handling, JSON input, and output. Their mathematical comparison contract is an integer/list certificate with a canonical closed guarded public goal. The compact module does not itself validate that public grammar or the entire data alphabet. The pinned compact documentation explicitly makes this distinction [2].

The strict module adds both checks. It rejects booleans, floats, strings, objects, and null anywhere in the certificate, including unused prefixes. It also checks the canonical guarded Eq/D public grammar. Scope checking remains in the kernel. It permits an unused, well-typed prefix of instructions, as does the comparison protocol. Its 1183 bytes are reported separately; there is no claim that it improves every existing validated checker.

2 The actual refinement

The only change to the byte-control kernel is its goal-selection expression. Write

$$C \equiv (m, a) = \big(s \texttt{ and } [s, u] \texttt{ or } [t, t, t],\ i(t, u)\big), \qquad R = i(t, s).$$

Here C is a Python Boolean obtained by structural comparison, and i is the scope-sensitive opener. The previous expression was

```
a = C and R or a
```

and the replacement is

```
a = (a, R)[C]
```

The actual module inlines C and R. Its size falls because the normalized indexed expression is shorter in bytes. This is not an arbitrary rewrite of Python's Boolean-selection idiom: false values and the additional eager evaluation have to be addressed.

2.1 Root shape and false results

For fixed current floor k, the opener is

```
i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d
    else d % 2 and i(t, d=3)
```

The proof in this section is over finite trees whose leaves are integers and whose internal nodes are lists. The strict module enforces that alphabet.

Lemma 2.1 (Root-shape lemma)**.** *Suppose $i(t, s, 0)$ and $i(t, u, 0)$ both terminate successfully. If t is a list, their results are lists with the same outer arity as t. If t is an integer, the two results are equal and independent of the image. Consequently, if either result is false in Python, the results are equal: they are both the empty list or both the same zero.*

Proof. On a list, the first branch constructs one output child per input child, without changing the outer list. A nonempty list is true irrespective of its children. On an integer at depth zero, the interval branch returns that integer when $k \le t < 0$; otherwise the factor 0 mod 2 returns zero. Neither scalar case reads the image. The only false outer list is the empty one. This proves each assertion. □

Lemma 2.2 (Equality of the selected goal)**.** *Whenever the new indexed expression completes, it selects the same goal as the old Boolean expression evaluated from the same state.*

Proof. If C is false, both select a. If C is true, the old match establishes $a = i(t, u, 0)$. For a true replacement R, both expressions select R. For a false replacement, Lemma 2.1 gives $R = i(t, u, 0) = a$, so the old fallback and the new selected value are still equal. No assumption that all intermediate goals are nonempty formulas is needed. □

2.2 A successful trace is not enlarged

Theorem 2.3 (Successful-trace inclusion)**.** *On every finite integer/list certificate accepted by the new compact program, the pinned byte-control program also accepts. Along the corresponding executions they consume the same instructions and have the same current goal, context, and scope state at each common checker entry.*

Proof. The new expression evaluates R eagerly before evaluating C. The opener is pure: it creates lists, performs arithmetic comparisons and recursive calls, and neither mutates its input nor consumes an instruction. On a successful new evaluation all operations needed by the old expression are therefore available with the same values. Lemma 2.2 proves equality of the selected goal.

The remainder of the instruction is unchanged. Both executions check the same major premise recursively, compute the same supplied hypothesis, decrement the same local floor, and form the same continuation goal and context. The expression `g + [new_hypothesis]` allocates a new list; it does not mutate a parent's context or the default list. In particular, local hypotheses from a recursively checked major do not escape into its caller. Induction over the finite successful execution tree gives the claimed correspondence and the same final output. □

The converse assertion for all raw certificates is deliberately absent. The extra eager operation can reject an old trace whose otherwise unused field is malformed. This is a restriction of raw certificates, and it does not introduce an extra accepted theorem.

The supplied `fixtures/eager_control_only.json` makes the distinction concrete. Before an ordinary positive computation proof it inserts the unused packet `[0, 3, [0], [5, 0]]`. Both control programs accept the certificate, while both refined programs reject it: the new eager opening tries the out-of-scope image 3 at numerical root depth 3. The public goal itself is valid and retains its original canonical proof. Thus raw certificate equality would be a false claim even with a validated public goal; equality of the recognized theorem sets is the relevant assertion.

2.3 Canonical completeness survives the extra evaluation

Lemma 2.4 (Total opening with a valid image)**.** *Fix k. If the numerical-root check $i(s, d = 3)$ terminates successfully, then $i(t, s, d)$ terminates for every finite integer/list tree t and every integer initial depth d.*

Proof. Induct on t. A list makes finitely many calls on proper subtrees. At an integer leaf the result is that integer, zero, or the fixed successful numerical-root check of s. No recursive call continues into a newly inserted image with the same substitution argument. Thus the structural induction terminates even when an ignored annotation has negative depth. □

Theorem 2.5 (Canonical completeness preservation)**.** *Every theorem of the pinned guarded public presentation that has its intended canonical certificate has a certificate accepted by the new compact program. Consequently the new and old programs recognize the same theorems on that public language, with the inherited certificate compiler.*

Proof. Keep a canonical source certificate and standardize unused fields. A genuine transport or closure-elimination packet already evaluates its selected replacement in the old successful execution. Alternatively, its source is a valid outer numerical term, so Lemma 2.4 applies. A canonical computation packet also has a valid source (zero or its well-scoped predecessor term). For an introduction packet, set its unused source field to zero. Its zero major cannot accidentally match the nonempty reconstructed elimination major, and its introduction branch is unchanged. For a D-elimination packet set all three inert fields to zero. Its unary major cannot match the ternary reconstructed major; the exported closure is unchanged, and the eager opener on zero terminates.

There is also a direct check for canonical numerical bodies and witnesses: their node arities are one and three. An opening that starts at formula depth zero stays at even depth, so it never inspects the substitution image. Thus the additional operation is total on every intended packet. Apply the same finite-execution induction as in Theorem 2.3. □

This is a refinement of an existing calculus. Its whole-domain bi-interpretation is inherited. A new proof of that arithmetic equivalence is not being inferred merely from passing a finite test suite.

3 The inherited mathematical presentation, explicitly

To specify exactly which arithmetic strength is intended, this section states the ordinary native presentation behind the compact public codec. It is the existing Choice-DI presentation of the repository [4, 1]. It is not a new axiom inventory.

3.1 Terms, formulas, and semantics

There is one numerical sort. The finite terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u in the body h is bound. Each fixed body is a staged first-order function description built from earlier descriptions. Numerical variables do not range over functions, and there is no uniform object-language evaluator of all primitive-recursive codes.

Use formulas

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A,$$

with readings

$$\mathsf{Q}_y B := B(0) \to \forall y\, B(y), \qquad \mathsf{D}_{x,y} B := \exists x (B(x, 0) \to \forall y\, B(x, y)).$$

The designated native contradiction is $0 = S0$. Contexts are finite reusable lists of formulas; all substitutions avoid capture.

3.2 The complete rule inventory

Besides the two displayed computation schemes, there are six inference patterns. They are listed separately to make every primitive explicit.

$$\Gamma \vdash I(0, a; u.h) = a, \qquad \text{I0},$$
$$\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u], \qquad \text{IS}.$$

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \text{Hyp} \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{Tr}$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE}$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B \quad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}\ \text{DE}$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}\ \text{DI}_{\text{case}}$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)}\ \text{DI}_{\text{progress}} \qquad (E \text{ a literal equation}).$$

In DE, b is fresh for Γ, the conclusion, and the passive parameters of B. In each introduction, z is fresh for Γ, t, and the passive matrix parameters. The two bound variables of D are distinct. The

transport template F is any fixed native formula template. The displayed local assumptions are discharged.

No separate successor-injectivity, successor-separation, reflexivity, ordinary logical, or additional induction axiom is primitive here. The equivalent current kernel uses direct progress introduction of $\mathsf{Q}_y E$; the unused-witness construction converts between the two formulations. The compact checker applies further established public-language and certificate transformations, discussed in Section 6.

3.3 Equality, cases, and equation induction

Put $r = I(0, t; u.0)$. The computation $r = t$, used as both transport premises with template $F(v) \equiv (v = t)$, gives $t = t$. Ordinary equality templates then give symmetry, transitivity, and numerical congruence.

Progress introduction with an unused witness coordinate gives $\mathsf{D}_{v,y} E(y)$. Its elimination supplies the conditional closure of E. Using the base $E(0)$ and QE therefore derives contextual equation induction:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

The corresponding case introduction, ignoring its local assumption, derives numerical cases for every native formula:

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}.$$

Both statements have the usual fresh-z condition. Induction remains restricted to literal equations. Allowing progress at arbitrary compound matrices would yield arbitrary-formula induction and would change the intended arithmetic strength.

4 Recovering all intuitionistic logical operations

The following constructions are inherited from the Choice-DI development [4]. They explain why the economical native formula grammar still presents full first-order intuitionistic logic.

First remove an auxiliary $\mathsf{Q}_y B$ by the interderivable $\mathsf{D}_{u,y} B(y)$ with unused u. Define the numerical selector

$$C(v, a, b) := I(v, a; u.b), \qquad C(0, a, b) = a, \quad C(Sz, a, b) = b.$$

For Eq/D formulas define a gate

$$G_v(s = t) := C(v, 0, s) = C(v, 0, t), \qquad G_v(\mathsf{D}_{x,y} B) := \mathsf{D}_{x,y} G_v B,$$

renaming bound variables away from v. Equality and binder congruence give $\vdash G_0 A$ and $G_{Sz} A \dashv\vdash A$. Padding

$$L(A) := \mathsf{D}_{u,y} G_y A$$

with unused fresh u is interderivable with A: introduce by cases in one direction, and open, supply the true zero gate, and specialize at one in the other.

Every Eq/D formula is a chain of paired binders ending in an equation. To construct a selector $M_v(A, B)$, pad the shallower operand until both have the same number of binders, align fresh bound names, and replace their terminal equations $s = t$ and $p = q$ by

$$C(v, s, p) = C(v, t, q).$$

The control is lifted under the aligned binders. The gate and padding proofs establish the endpoint equivalences $M_0(A, B) \dashv\vdash A$ and $M_{Sz}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. The computations $p(1) = 1$ and $p(2) = 0$ suffice for the macros

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned}$$

Every new binder is fresh and the displayed u is unused.

Implication introduction uses its zero row as the assumption and its successor row as the consequent. Elimination opens its closure and specializes at one. Existential introduction supplies its witness; elimination opens at a fresh witness and removes the gate at one. Disjunction injections use witnesses zero and one, while elimination uses numerical cases on the selector control. The conjunction has a provable zero row, and its rows at one and two recover B and A. Numerical cases prove its introduction rule.

The universal macro's first conjunct is essential: it supplies the zero instance erased by the other conjunct's gate. The paired conjunct supplies all successor instances. Numerical coverage then supplies the instance at an arbitrary term. Coverage itself is derived by numerical cases:

$$(x = 0) \vee_* \exists_* v\, (x = Sv).$$

At zero use the first injection; at a successor use the second injection with its evident witness and reflexivity.

Under $0 = 1$, selector computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for all terms. Thus every equation follows. External induction on finite formula syntax, using case introduction, gives every native formula. This proves falsity elimination without introducing excluded middle for quantified formulas.

5 The inherited whole-domain bi-interpretation

5.1 Ordinary realization and the falsity bridge

Let T_I be intuitionistic first-order logic with actual equality and genuine falsity over the staged iterator signature. Its nonlogical axioms are I0 and IS, numerical coverage,

$$\forall x\, (x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = S0) \to \bot$, and contextual induction for literal equations. Separation is explicit: positive numerical equations alone admit a singleton structure. The native contradiction convention must not be mistaken for an ordinary first-order falsity axiom.

Interpret the native binders by their displayed ordinary meanings, using the whole domain $x = x$ and actual equality. Case introduction is sound by coverage, and progress introduction is sound by equation induction under its conditional base. The reverse interpretation uses the preceding logical macros. A crucial internal round trip is

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big).$$

Forwards, open the paired formula, reconstruct its conditional universal, and introduce the existential macro. Backwards, open the existential and use case introduction at that witness. The selector endpoints give the ordinary logical round trips; structural induction extends them to every finite formula. These are full formula comparisons, not merely agreement on closed equations.

5.2 The arithmetic dependency order

The scalar body does not see its recursion counter. Recovering all primitive-recursive functions therefore requires an internal arithmetic argument. The supplied reference sources give it in the following order [5, 6, 7, 8].

First, using computations, equations, cases, and equation induction, construct a scalar predecessor P and prove internally

$$P0 = 0, \qquad P(Sn) = n.$$

The construction uses a parity iterator, a three-state cycle, a ramp, a factor map, and a saturated counter. It does not assume pairing. Its final form is

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big),$$

where L is the comparison term constructed earlier in that proof. The complete inherited definitions and derivation are provided in `reference/foundations/scalar_predecessor.tex`. These identities hold at arbitrary internal arguments, not just at standard numerals.

Next define truncated subtraction

$$d(x, y) = I(y, x; u.Pu), \quad \mathrm{sg}(v) = C(v, 0, 1), \quad K(v) = C(v, 1, Pv).$$

The internally proved residual identity $d(Sa, b) = 1 \leftrightarrow a = b$ yields the actual-equality characteristic

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big), \qquad e(a, b) = 0 \leftrightarrow a = b,$$

with $e(a, b)$ always zero or one. In particular the equality being reflected is the original equality relation, not an assumed congruence or quotient.

Compile each fixed quantifier-free formula A to a Boolean-valued term b_A, using

$$\begin{aligned} b_{s=t} &= e(s, t), & b_{\perp} &= 1, \\ b_{A \wedge B} &= C(b_A, b_B, 1), & b_{A \vee B} &= C(b_A, 0, b_B), \\ b_{A \to B} &= C(b_A, b_B, 0). \end{aligned}$$

Induction on finite formula construction proves $b_A = 0 \leftrightarrow A$ and that b_A is zero or one. The implication clause uses decidability of its quantifier-free antecedent. Equation induction on $b_A(n) = 0$ then gives every quantifier-free induction instance, with retained side assumptions. This does not make quantified formulas decidable.

After quantifier-free induction is available, construct the pairing $\pi(a, b) = (a + b)^2 + a$. A bounded square-root search produces $q(n)$ with $q(n)^2 \le n < (q(n) + 1)^2$. Put

$$L_p(n) = d(n, q(n)^2), \qquad H_p(n) = d(q(n), L_p(n)).$$

Then $L_p(\pi(a, b)) = a$ and $H_p(\pi(a, b)) = b$. Surjectivity is unnecessary: pairing stores an internal state and does not replace the numerical domain.

For a fixed primitive-recursive description

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

define its scalar compilation by

$$W(u,\vec{p}) = \pi\big(SL_p(u), h^*(L_p(u), H_p(u), \vec{p})\big),$$
$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})\big),$$
$$f^*(n,\vec{p}) = H_p(V(n,\vec{p})).$$

Equation induction proves $L_p(V(n,\vec{p})) = n$, and the projection identities give both defining equations of f^*. Initial functions and composition translate directly. The compilation is external recursion on a fixed finite description, not an object-language universal evaluator.

5.3 Both composites are proved internally

Conversely, iPRA interprets each fixed scalar iterator by ordinary primitive recursion whose update ignores the counter. It proves the iterator equations, coverage, separation, and equation induction.

For two fixed iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every internal counter. The same argument applies to ordinary primitive recursion. Inducting externally over a finite function description shows that its double translation satisfies the original defining equations; internal uniqueness identifies the original and double-translated operations. This also supplies coherence when substituting parameters changes the staged spelling of an update.

Theorem 5.1 (Inherited arithmetic equivalence)**.** *The ordinary realization T_I is bi-interpretable with full first-order* iPRA*. Both interpretations have whole numerical domains and actual equality. The internally definable isomorphism for each numerical composite has graph $x = y$. Together with the logical round trips and preservation of falsity, the finite proof translations give equiconsistency.*

Proof. The two interpretations of fixed function symbols and axioms were described above. Internal uniqueness proves the numerical round trips; structural induction and the logical endpoint arguments prove the formula round trips. The explicit falsity bridge preserves contradiction in both directions. The graph $x = y$ is internally a total bijection preserving every interpreted operation. These facts are the inherited proof of the theorem, supplied in full source form with the archive [5, 8]. □

No claim is made that either consistent theory proves its own consistency. The new 513-byte implementation inherits this equivalence through the unchanged guarded public presentation and Theorems 2.3–2.5.

6 The compact codec is a separate layer

The mathematical rule inventory above is not a literal description of every raw JSON value accepted as an intermediate resource. The current compact program uses the following arity representation:

$$\mathsf{D}B \longleftrightarrow [B], \qquad s = t \longleftrightarrow [s,t], \qquad \mathsf{Q}B \longleftrightarrow [B,B,B].$$

Terms use 0, scalar coordinates, unary successor arrays, and ternary iterator arrays. The opener changes its depth by $5 - r$ at a list of arity r. For relevant arities $1, 2, 3$ these increments are $4, 3, 2$. Two-wide numerical slots and dummy slots make the one traversal implement the distinct logical and numerical binders.

The sorted reading of additional arrays is essential. High-arity formula padding denotes truth; inappropriate numerical shapes denote zero. Tripling a unary D major produces a ternary closure

resource. Tripling other nonempty major arrays produces administrative truth. These extra arrays are representation aliases, not new unrestricted logical symbols.

The current compact kernel restricts transport sources away from literal zero and chooses the introduction mode from the matrix root: progress at an equation and cases at a compound matrix. The pinned Lean development proves that the relevant restricted calculus agrees with the native kernel on its guarded public translation, and also proves that removing this public-language qualification would be wrong [1]. Public atomic matrices under D use the guard

$$I(y, 0; u.h) = I(y, 0; u.k).$$

The formula gate used in the logical constructions supplies the required translation to guarded formulas. An arbitrary native formula is not to be identified with an arbitrary raw administrative array.

The supplied older `inherited_arity_checker.tex` explains the sorted decoder and arithmetic-supply mechanism. Its historical packet interface has a choice of introduction modes and differs from the current program. Its old full-native completeness statement is therefore background, not a theorem transplanted verbatim onto the 513-byte module. The actual refinement theorem compares the pinned executable byte control with the new executable, under the current contract [2].

6.1 Complete compact program

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d
     else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = (a, i(t, s))[(m, a) == (s and [s, u] or [t] * 3, i(t, u))]
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
     i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or ~len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

An input is a JSON list whose last item is the goal; preceding four-field instructions are popped from right to left. Normal termination with exactly `ACCEPT\string \n` means acceptance. An exception means rejection. The byte comparison includes the complete program shown above. The mathematical algorithm is understood with unbounded resources; physical Python recursion and memory limits can reject long proofs.

6.2 Why context and phase changes need proofs

Two attractive source shortenings were rejected. Replacing an immutable context extension by in-place list extension would leak local hypotheses from a recursively checked major into its parent. Removing the scope counter by the entry identity $k = 1 - |g|$ also requires changing the phase order: the original program decreases k before the new context entry has been inserted. A direct textual replacement uses the old context length while checking the new eigenparameter and rejects valid proofs. Neither change is present in the delivered implementation.

The harmless extra opener in the byte refinement has a much smaller proof obligation: it changes no state. That is why its correctness can be established locally by the root-shape and totality lemmas.

7 A barrier to removing every supplied term

Several recent proposals remove the explicit witness from introduction; another restricts clause specialization to the newest parameter or a fixed numeral. Their naive combination does not preserve the intended native consequence relation. This is a review of the fixed-range obstruction already anticipated in the repository, not a new arithmetic breakthrough [11, 12].

Consider a clause calculus with introduction choosing the newest outer parameter, or zero in an empty scope. Suppose every allowed specialization term is a parameter, zero, or one. Its intended D reading is the one above. There is a different Boolean reading that validates those restricted rules and refutes a standard naming theorem.

Evaluate numerical terms in $\mathbb{N}$, assigning every ambient free parameter zero. Evaluate formulas recursively, with

$$[\![\mathsf{D}_{x,y}B]\!]_\rho = \big([\![B]\!]_{\rho[x:=0,y:=0]} \Rightarrow [\![B]\!]_{\rho[x:=0,y:=1]}\big).$$

Read a retained clause $[A; y.B(y)]$ as

$$[\![A]\!] \Rightarrow ([\![B(0)]\!] \wedge [\![B(1)]\!]).$$

An antecedent-free clause means the conjunction of those two instances.

Computation and equality remain valid because numerical terms have their ordinary values. Every permitted specialization term has value zero or one, so clause use is sound. In introduction, the selected witness and fresh eigenparameter both have value zero; either local assumption $B(0,0)$ or $B(0,z)$ has value $B(0,0)$, and the successor conclusion has value $B(0,1)$. Thus case and equation-progress introduction are sound. Ordinary elimination opens at the fresh value zero and adds the clause $B(0,0) \Rightarrow (B(0,0) \wedge B(0,1))$, which follows from the major.

An inferred equality discharge is also sound. If its zero row is literally true and its first row is $b = t$, the major forces t to have value zero, the same as the opened b. Substitution preserves the value of the exported formula. In the default case it is identity substitution.

But the closed native formula

$$N_1 := \mathsf{D}_{x,y}\big(C(y,0,x) = C(y,0,1)\big)$$

has false counterreading $(0 = 0) \Rightarrow (0 = 1)$. In the intended calculus it is a theorem: choose witness one, making the two terms identical. Therefore the combined restrictions lose an intended closed theorem.

This establishes failure of the intended native consequence equivalence. It is not an impossibility proof for every conceivable different bi-interpretation of the restricted formal system.

8 Executed evidence and its limits

The supplied `reproduce.py` runs with the Python standard library and was executed under CPython 3.12.14. Its complete results are in `evidence/results.json`. The checks are:

- Twenty-one pinned regression certificates, including computation, transport, compound introduction, implication elimination, used induction hypotheses, zero-source transport, and scope/context attacks. Each is run through the compact control, compact candidate, strict control, and strict candidate, under both ordinary and optimized compilation: 168 outcomes.

- Twelve actual command-line subprocess checks of accepting and rejecting inputs, including Python's `-O` mode. These check the input/output contract independently of the in-process harness.
- 1200 deterministic random integer/list certificates and 600 mutations of known positive certificates. No new acceptance absent from the control was observed. There were 75 candidate acceptances in the random set and 14 in the mutation set; these counts include administrative test goals and are not counts of new arithmetic theorems.
- Twelve additional malformed-input or invalid-goal checks for the strict candidate. Noninteger/list unused prefixes are rejected.
- An explicit control-only certificate verifies the strict raw-trace inclusion boundary: both controls accept and both refinements reject it, while its public goal still has the original accepted canonical proof.
- Reproduction of all six full-module measurement rows in Section 1.

All assertions passed. One pinned accepted fixture deliberately has an unguarded auxiliary goal; the compact comparison accepts it and the strict public checker rejects it, as specified. The other regression expectations are preserved.

These are finite implementation checks. They neither prove universal soundness nor establish bi-interpretability. The universal claims rely on the written refinement proof and the identified inherited mathematical theorems. No new Lean or Rocq development was compiled for this report.

9 Further research questions

1. **Can a mathematical change beat every coordinate?** Find a complete module with fewer than 256 tokens, fewer than 278 AST nodes, and fewer than 516 normalized bytes, using a fixed, stated input contract. A single module would be stronger evidence than three separate profiles. The present 513-byte program does not do this.
2. **Can validation become cheaper together with the kernel?** The public grammar and the integer/list alphabet have visible costs. An improvement that shares a proved invariant between parsing, binding, and proof checking could be more useful than another isolated byte saving. Its soundness statement must cover the full accepted input language.
3. **Can scope state be eliminated with a smaller phase-aware design?** Context length determines the local scope at checker entry, but the continuation temporarily uses the next scope. A semantics-preserving design must account for that transition and for immutable branch contexts.
4. **Can introduction and specialization both be automatic without a costly enumerator?** The finite-target counterreading rules out a natural combination. Enumerating terms through dummy scopes can avoid a finite range, but the decoder, scope validation, and proof-padding construction must all be counted. That is presently an interface tradeoff, not a smaller trusted arithmetic basis.
5. **Can the refined checker itself be formally connected to the Lean kernel?** The repository formalizes substantial native-calculus equivalences. An end-to-end theorem about the exact JSON parser, opener, administrative aliases, and standalone program would remove a remaining implementation-to-mathematics boundary. The local root-shape lemma is a small suitable starting point.

6. **Can an alternative arithmetic signature simplify the internal bridge and the checker simultaneously?** A recursor with direct access to its counter simplifies predecessor, but existing normalized presentations pay for their normalizer and syntax. Both source size and internal round-trip proofs need to be measured together.

A Using the archive

The archive contains this report as LaTeX and PDF, the two candidate checkers, matched controls, pinned arithmetic reference sources, all regression fixtures, the reproducibility script, evidence, claim and provenance files, and checksums. The mathematical sources are retained with attribution and the inherited license.

From the extracted directory:

```
python3 reproduce.py
python3 code/checker_strict.py < fixtures/computation_zero_local.json
pdflatex -interaction=nonstopmode -halt-on-error ipra_frontier_review.tex
pdflatex -interaction=nonstopmode -halt-on-error ipra_frontier_review.tex
python3 verify_manifest.py
```

The main report compiles directly; the inherited fragment sources are reference material and are not separate standalone LaTeX documents.

B The validated standalone program

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d
     else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = (a, i(t, s))[(m, a) == (s and [s, u] or [t] * 3, i(t, u))]
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
     i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or ~len(a) % 2) and k)], k)

def data_ok(a):
    return type(a) is int or (type(a) is list and all(map(data_ok, a)))

def formula(a, numerical=False):
    if type(a) is int:
        return numerical and (a == 0 or (a >= 3 and a % 2 == 1))
    size = len(a)
    return size in ([1, 3] if numerical else [1, 2]) and all((formula(b, numerical or size == 2)
    for b in a)) and (numerical or size == 2 or len(a[0]) == 1 or all((b[:2] == [5, 0] for b in a
    [0])))
p = json.load(open(0))
if type(p) is not list or not data_ok(p):
    raise ValueError('The certificate must be a list containing only integers and lists')
q = p.pop
a = q()
if not formula(a):
    raise ValueError('The goal must be a canonical closed guarded Eq/D formula')
c(a)
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt, revision `62396b06020f794fa6dc01b4025a6a5db9d4d28a`, project and current README. Pinned local snapshot: `reference/README_pinned.md`.

[2] ProveIt, *canonical_recursors/compact*, pinned compact comparison package. The 516-byte control and `ARGUMENT.txt` are reproduced.

[3] ProveIt, *kernel_refinement*, pinned package. Sources of the 256-token and 278-node comparison modules.

[4] ProveIt, *inherited_choice_basis.tex*, certified_zero reference collection. Reproduced in `reference/foundations/`.

[5] ProveIt, *inherited_ordinary_bridge.tex*, same pinned reference collection. Explicit ordinary realization and whole-domain bi-interpretation.

[6] ProveIt, *scalar_predecessor.tex*, same pinned reference collection. Internal scalar predecessor construction.

[7] ProveIt, *equality_reflection.tex*, same pinned reference collection. Reflection of actual equality and quantifier-free induction.

[8] ProveIt, *primitive_recursion.tex*, same pinned reference collection. Pairing, compiler, and both internal numerical composites.

[9] ProveIt, *newest_parameter*, pinned package. Introduction at the newest parameter and the arithmetic dependency audit.

[10] ProveIt, *conditional_copy*, pinned package. Automatic introduction combined with inferred discharge.

[11] ProveIt, *constructed_computation*, pinned package. Includes fixed-range obstructions to simultaneous witness and specialization restrictions.

[12] ProveIt, *deterministic_specialization*, pinned package.